

CRDTs and Databases: A Hands-On Tutorial

Part 1 — Conflict-free Replicated Data Types

Paulo Sérgio Almeida Nuno Faria José Pereira

INESC TEC and University of Minho

VLDB 2026



INESCTEC



Universidade do Minho
uminho.pt

From sequential data types to CRDTs

Operation-based CRDTs

Pure operation-based CRDTs

State-based CRDTs

Delta State CRDTs

Comparison of approaches

Data abstractions

Functional abstractions

- ▶ functions, procedures
- ▶ Fortran, Lisp, Algol

Data abstractions

- ▶ objects, abstract data types
- ▶ Simula, CLU

Ingredients for data abstractions

- ▶ Algol: procedures, blocks
- ▶ AED-1: records (then called *plexes*)
- ▶ Simula: classes (records with procedures, or generalized Algol blocks)
- ▶ CLU: hidden representation

Ross and Feldmann (1964), **Verbal and graphical language for the AED system: A progress report**

Dahl and Nygaard (1966), **SIMULA - an ALGOL-based simulation language**

Liskov and Zilles (1974), **Programming with Abstract Data Types**

Sequential data types

- ▶ Most common abstractions in libraries; e.g., Set, Map
- ▶ Lot of research effort in imperative implementations
 - immensely useful for the “real world”
- ▶ Relatively easy to prove correctness
 - preconditions, postconditions, invariants
 - at least when no aliasing involved
- ▶ Hard to implement efficiently in functional languages
 - no destructive updates + persistence
 - amortization achieved with lazy evaluation (Okasaki)

Hoare (1969), **An Axiomatic Basis for Computer Programming**

Hoare (1972), **Proof of Correctness of Data Representations**

Okasaki (1999), **Purely functional data structures**

Concurrent data types

- ▶ Objects usable by concurrent clients
 - Hansen *shared classes*, inspired by Simula
 - Hoare *monitors*
- ▶ Enforce mutual-exclusion during operation execution
 - allow *atomic objects*, easy to reason about
- ▶ Blocking mid-operation very useful, but complicates matters
 - e.g., bounded-buffer
 - await primitive (Hansen), condition variables (Hoare)
 - not common with distribution (e.g., distributed lock)

Hansen (1973), **Operating System Principles**

Hoare (1974), **Monitors: An Operating System Structuring Concept**

Lock-free and wait-free data structures

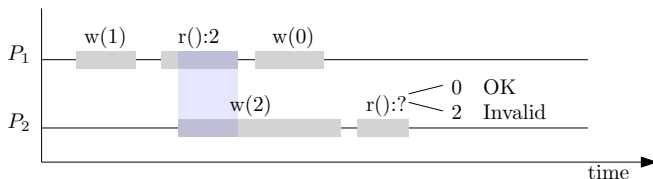
- ▶ Several operations may progress concurrently
- ▶ To allow performance and/or fault tolerance
- ▶ Wait-free: operations complete in finite number of steps
- ▶ Emphasis on multiprocessors, not distributed systems
- ▶ How to express correctness criteria?

Linearizability

Execution linearizable if

- (1) equivalent to some sequential execution
- (2) respects “real time” for non-overlapping operations

Register example:



*“Linearizability provides the illusion that each **operation** applied by concurrent processes takes effect **instantaneously** at some point **between** its **invocation** and its **response**”*

Replicated state machines

Recipe:

- (1) Replicate state on several nodes
- (2) Same deterministic state machine at each node, $S' = F(S, I)$
- (3) Agree on global total order of inputs from all nodes
 - consensus, difficult under faults
- (4) Apply totally ordered inputs at each node

Blockchains follow this approach, many use some variant of PBFT

Sequential datatype + replicated state machine => replicated datatype

Problem solved?

Lamport (1978), **Time, Clocks, and the Ordering of Events in a Distributed System**
Pease, Shostak, and Lamport (1980), **Reaching Agreement in the Presence of Faults**
Castro and Liskov (1999), **Practical Byzantine Fault Tolerance**

The CAP theorem

We can have at most two of:

- ▶ strong Consistency (linearizability)
- ▶ Availability
- ▶ Partition tolerance

CP systems

- ▶ aim for linearizability
- ▶ may become unavailable under network partitions
- ▶ high response times in wide area

AP systems

- ▶ operations remain available, even during partitions
- ▶ low response times even in wide area

Partitions cannot be avoided, we can design AP or CP systems

Brewer (2000), **Towards robust distributed systems (abstract)**

Gilbert and Lynch (2002), **Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services**

AP systems aim for Causal Consistency + Convergence

Causal Consistency (CC)

- ▶ processes may see different orders, consistent with a global **happens-before** partial order that contains **process orders** and operation **visibility**

$$\text{hb} \doteq (\text{po} \cup \text{vis})^+$$

- ▶ a visible to b if effect of a visible to process performing b
- ▶ CC: each update u in potential causal past of operation o is visible to o :

$$u \xrightarrow{\text{hb}} o \Rightarrow u \xrightarrow{\text{vis}} o$$

Convergence

- ▶ processes that see same set of updates have equivalent state
- ▶ regardless of the order in which they become visible

Eventual consistency: eventual visibility + convergence

Spatial scalability

CC + Convergence

- ▶ exhibits “mechanical sympathy” regarding our universe
 - model suits the “machine”
- ▶ we can achieve it while being as fast as physically possible
- ▶ scales spatially

Linearizability

- ▶ is contra naturam; wide mismatch between model and “machine”
 - would suit an Aristotelian universe, with infinite light speed
- ▶ we must delay outcome to achieve it
- ▶ response time degrades with spatial span

CC + Convergence: suitable for truly global large scale systems

Optimistic replication

- ▶ Replicas locally available for queries and updates
- ▶ Updates propagated asynchronously
- ▶ Achieves availability and low latency

What about possible replica divergence?

- ▶ relax ordering, but only for commutative operations (Lazy replication)
- ▶ apply ad hoc client-provided merge procedures (Bayou)

Ladin et al. (1992), **Providing High Availability Using Lazy Replication**

Terry et al. (1995), **Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System**

Conflict-free Replicated Data Types

- ▶ Provide operations, like standard abstract datatypes
- ▶ Each datatype object replicated and accessed locally
 - Mutator operations update state
 - Query operations look at state and return result
- ▶ Information propagated to other replicas asynchronously
- ▶ Object highly available even under partitions
- ▶ Conflicts are dealt with semantically: concurrent spec of datatype

Not limited to commutative operations; no application-specific ad hoc code

ADTs vs objects

ADTs

existential types
can access multiples instances
efficient binary operations
different implementations cannot mix

Objects

recursive higher order functions
access only self state
slow/impossible binary operations
mix multiple implementations

Why do we care?

- ▶ Only with full replication are distributed ADTs viable
 - we can directly access only local node state
 - but we may shard the set of objects across several nodes
- ▶ CRDTs normally are really “Conflict-free Replicated Objects”

Operations cannot access multiple instances, only self state

Operation-based vs state-based approaches

Operation-based approaches

- ▶ propagate information about operations
- ▶ use a reliable messaging algorithm for propagation
- ▶ need ordering guarantees (typically causal)
- ▶ special case: *pure* operation-based approach

State-based approaches

- ▶ propagate replica states
- ▶ reconcile (merge) replica states
- ▶ anti-entropy by opportunistic, “background” communication
- ▶ can be made more incremental by delta-state approach

From sequential data types to CRDTs

Operation-based CRDTs

Pure operation-based CRDTs

State-based CRDTs

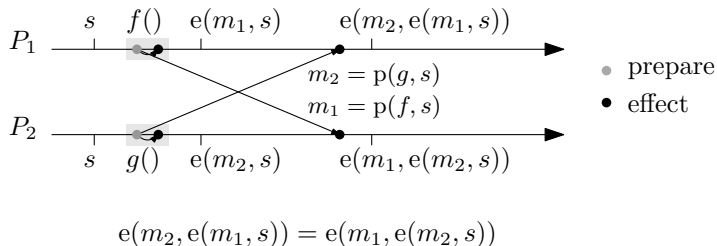
Delta State CRDTs

Comparison of approaches

Operation-based CRDTs

- ▶ Core concept
 - send operations, not state, to other replicas
 - operations applied at each replica
- ▶ Uses reliable causal broadcast
 - ensures exactly-once for non-idempotent operations
 - respects order of causally dependent operations
- ▶ What about non-commutative operations?
 - applied in different orders would lead to divergence
 - dealt with by sending more than just the operation

Standard execution model of operation-based CRDTs



Prepare performed where operation invoked

- ▶ looks at state and operation
- ▶ no side effects (on abstract state)
- ▶ result disseminated with reliable causal broadcast

Effect applied at each replica, upon delivery

- ▶ designed to be commutative for concurrent operations
- ▶ assumes immediate self-delivery on sender replica

Simple CRDTs – with only commutative operations

GCounter

CRDT state:

$n : \mathbb{N} = 0$

query value() : $\mathbb{N}$
return n

update inc()
prepare
return inc
effect inc
 $n \leftarrow n + 1$

PNCounter

CRDT state:

$v : \mathbb{Z} = 0$

query value() : $\mathbb{Z}$
return v

update inc()
prepare
return inc
effect inc
 $v \leftarrow v + 1$
update dec()
prepare
return dec
effect dec
 $v \leftarrow v - 1$

Grow-only set (GSet(E))

CRDT state:

$s : \mathcal{P}(E) = \emptyset$

query elements() : E
return s

query contains($e : E$) : $\mathbb{B}$
return $e \in s$

update add($e : E$)
prepare
return (add, e)
effect (add, e)
 $s \leftarrow s \cup \{e\}$

- ▶ Standard sequential datatype for replica state
- ▶ Not even FIFO needed for convergence, just exactly-once delivery
- ▶ Causal delivery normally used to achieve causal consistency

CRDTs with non-commutative operations

- ▶ Example: set with add and remove operations:

$$\text{add}(v, \text{rmv}(v, s)) \neq \text{rmv}(v, \text{add}(v, s))$$

- ▶ Effect cannot be defined as simply applying operation
 - different delivery orders for concurrent operations
 - would lead to divergence
- ▶ Effect must be defined to be commutative for concurrent operations
- ▶ But how we define what is supposed to happen?

Defining concurrent semantics

- ▶ We want to preserve sequential semantics, for sequential executions
- ▶ We must define how to handle conflicts, for concurrent invocations
- ▶ Example: given concurrent add and remove, which will “win”
 - add wins
 - remove wins
 - LWW, using totally ordered arbitration
- ▶ Semantics usually defined resorting to causal past
 - query depends on updates visible to it, ordered under *happens-before*
- ▶ Examples:
 - counter: the value is the size of the set of visible increments
 - add-wins set: an element is present given a visible add without a remove in its future

Observed-cancel semantics

- ▶ An elegant concept: act on observed (visible) operations
 - with causal delivery, cancel operations in causal past
 - acts upon closed, well defined set of ops
- ▶ Example: observed-remove set
 - has add and remove operations
 - remove only cancels adds visible to it
 - concurrent adds will not be canceled; will “win”
- ▶ Example: observed-reset counter
 - has inc and reset operations
 - resets cancel observed incs
 - allows sample-and-reset pattern

If some operation cancels others, do it only to observed operations

Observed-remove set (ORSet $\langle E \rangle$), optimized

types:

$\mathbb{I}$, set of replica identifiers

CRDT state:

$m : E \rightarrow \mathcal{P}(\mathbb{I} \times \mathbb{N}) = \emptyset$

$c : \mathbb{N} = 0$, auxiliary state

query elements() : E

return $\text{dom } m$

query contains($e : E$) : $\mathbb{B}$

return $m[e] \neq \emptyset$

update add($e : E$)

prepare

$c \leftarrow c + 1$

return (add, e , (i , c), $m[e]$)

effect (add, e , d , r)

$m[e] \leftarrow m[e] \setminus r \cup \{d\}$

update remove($e : E$)

prepare

return (remove, e , $m[e]$)

effect (remove, e , r)

$m[e] \leftarrow m[e] \setminus r$

- ▶ Algorithm for replica $i \in \mathbb{I}$, assuming causal delivery
- ▶ Map stores only non-empty sets, returning $\emptyset$ for unmapped keys
- ▶ Remove only cancels adds visible to it; concurrent adds will “win”

Beyond sequential semantics and API

- ▶ CRDTs attempt equivalence to sequential execution, if possible
- ▶ Some CRDTs exhibit behavior not possible by any sequential execution
 - example: observed-remove set
- ▶ In some CRDTs the interface itself is different from sequential datatype
 - example: multi-value register, made popular by Dynamo

DeCandia et al. (2007), **Dynamo: amazon's highly available key-value store**

Multi-value register (MVReg $\langle E \rangle$)

types:

$\mathbb{I}$, set of replica identifiers

CRDT state:

$s : \mathcal{P}(E \times (\mathbb{I} \times \mathbb{N})) = \emptyset$

$c : \mathbb{N} = 0$, auxiliary state

query read() : $\mathcal{P}(E)$

return $\{e \mid (e, _) \in s\}$

update write($e : E$)

prepare

$c \leftarrow c + 1$

let $r = \{d \mid (_, d) \in s\}$

return (write, ($e, (i, c)$), r)

effect (write, v, r)

$s \leftarrow \{(e, d) \in s \mid d \notin r\} \cup \{v\}$

- ▶ Algorithm for replica $i \in \mathbb{I}$, assuming causal delivery
- ▶ A read returns set of most recent concurrent writes
- ▶ A write overwrites the set into a singleton where invoked

From sequential data types to CRDTs

Operation-based CRDTs

Pure operation-based CRDTs

State-based CRDTs

Delta State CRDTs

Comparison of approaches

Pure op-based CRDTs

- ▶ A more restrictive kind of op-based CRDTs
 - General op-based can depart too much from op-based spirit
- ▶ In **pure** op-based CRDTs
 - prepare simply returns operation, ignoring current state
 - messages contain only operation (including arguments)
- ▶ Given operation o and state s :

$$\text{prepare}(o, s) = o$$

Pure implementations of commutative datatypes

- ▶ In commutative datatypes operations can be applied in any order:

$$f(g(s)) = g(f(s))$$

- ▶ CRDT spec based on sequential spec
- ▶ Effect is simply applying the operation:

$$\text{effect}(o, s) = o(s)$$

- ▶ GCounter, PNCounter, GSet are pure

Pure implementations of non-commutative datatypes

- ▶ Obtained by using partially ordered Log (PO-Log)
- ▶ Using **Tagged Causal Broadcast** (TCB) middleware:
 - delivers respecting causal order
 - provides **partial ordered timestamp**, when delivering message
 - provides **causal stability** information

Pure CRDTs based on partially ordered Log

$$\begin{aligned} S &= T \rightharpoonup O & s^0 &= \{\} \\ \text{prepare}(o, s) &= o \\ \text{effect}(o, t, s) &= s \cup \{(t, o)\} \\ \text{eval}(q, s) &= [\text{datatype-specific query function over PO-Log}] \end{aligned}$$

- ▶ State is a **PO-Log**: partially ordered log of operations
 - a map from timestamps to operations
- ▶ Effect adds entry (t, o) delivered by TCB to PO-Log
- ▶ Queries defined over PO-Log

Naive, but starting point for subsequent optimization

PO-Log based observed-remove (add-wins) set

$$\begin{aligned} S &= T \rightarrow O & s^0 &= \{\} \\ \text{prepare}(o, s) &= o & & (o \text{ either } [\text{add}, v] \text{ or } [\text{rmv}, v]) \\ \text{effect}(o, t, s) &= s \cup \{(t, o)\} \\ \text{eval}(\text{elements}, s) &= \{v \mid (t, [\text{add}, v]) \in s \wedge \nexists (t', [\text{rmv}, v]) \in s \cdot t < t'\} \end{aligned}$$

- ▶ Add wins over a concurrent remove on same element
- ▶ Only query is datatype specific
- ▶ Query mimics specification over partial order of operations

Unoptimized pure-op CRDTs are essentially runnable specifications

Causal stability

Causal Stability: Message with clock t is *causally stable* at node i when all messages subsequently delivered at i will have clock $t' \geq t$

- ▶ Message causally stable: no more concurrent messages delivered
 - promise about future deliveries; part of middleware API
- ▶ Causal stability stronger than classic message stability (Birman)
 - message stable when received by all nodes; middleware implementation issue

Semantically based PO-Log compaction

- ▶ PO-Log based CRDTs made efficient through PO-Log compaction
- ▶ Compaction is datatype dependent, according to specification
- ▶ Made possible by information provided by TCB

Information	Optimization
Causality	prune PO-Log after each operation delivered
Causal stability	discard timestamps of causally stable operations discard some causally stable operations

From sequential data types to CRDTs

Operation-based CRDTs

Pure operation-based CRDTs

State-based CRDTs

Delta State CRDTs

Comparison of approaches

State-based conflict resolution

- ▶ State-based approach propagates states
- ▶ If self state and received state conflict, must be merged
- ▶ In classic optimistic replication merged in some ad hoc way
- ▶ Conflict resolution (merge) should be
 - deterministic: result as a function of inputs
 - obviously a commutative function
 - associative: same result when merging in different orders
 - monotonic: merging with “newer” state produces “newer” state

Mathematical concept of *join-semilattice* is the solution

Partially ordered sets (posets)

- ▶ A partially ordered set (poset), has a binary relation $\sqsubseteq$:
 - (reflexive) $p \sqsubseteq p$
 - (transitive) $o \sqsubseteq p \wedge p \sqsubseteq q \Rightarrow o \sqsubseteq q$
 - (anti-symmetric) $p \sqsubseteq q \wedge q \sqsubseteq p \Rightarrow p = q$
- ▶ Two unordered elements are called concurrent.
 - (concurrent) $p \parallel q \Leftrightarrow \neg(p \sqsubseteq q \vee q \sqsubseteq p)$
- ▶ Some posets have a *bottom* ($\perp$), an element smaller than any other:

$$\forall p \in P . \perp \sqsubseteq p$$

Join-semilattices

- ▶ An upper bound of S is some u greater than any element in S

$$\forall s \in S \cdot s \sqsubseteq u$$

- ▶ If the *least upper bound* of S exists, it is called the *join* of S , denoted $\sqcup S$
- ▶ For two elements, $\sqcup\{p, q\}$ is denoted by $p \sqcup q$
- ▶ Poset P is a join-semilattice if $p \sqcup q$ is defined for any pair p and q in P
- ▶ Properties of join:
 - (idempotent) $p \sqcup p = p$
 - (commutative) $p \sqcup q = q \sqcup p$
 - (associative) $p \sqcup (q \sqcup r) = (p \sqcup q) \sqcup r$

Examples and non-examples of join-semilattices

► Some join-semilattices:

- natural numbers $\mathbb{N}$: $\sqsubseteq = \leq$; $\sqcup = \max$; $\perp = 0$
- booleans $\mathbb{B}$: $\text{False} \sqsubseteq \text{True}$; $\sqcup = \vee$; $\perp = \text{False}$
- any totally ordered set (*chain*)

► Some posets which are not join-semilattices:

- unordered posets (*antichain*)
 - all elements being incomparable
- strings under prefix ordering (e.g., $\text{small} \sqsubseteq \text{smallest} \parallel \text{smaller}$)
 - all concurrent elements are not joinable

Lattice compositions

Cartesian product $A \times B$

$$(a_1, b_1) \sqsubseteq (a_2, b_2) = a_1 \sqsubseteq a_2 \wedge b_1 \sqsubseteq b_2$$

$$(a_1, b_1) \sqcup (a_2, b_2) = (a_1 \sqcup a_2, b_1 \sqcup b_2)$$

(join-semilattices A and B)

Lexicographic product $A \boxtimes B$

$$(a_1, b_1) \sqsubseteq (a_2, b_2) = a_1 \sqsubset a_2 \vee (a_1 = a_2 \wedge b_1 \sqsubseteq b_2)$$

$$(a_1, b_1) \sqcup (a_2, b_2) = \begin{cases} (a_1, b_1) & \text{if } a_2 \sqsubset a_1 \\ (a_2, b_2) & \text{if } a_1 \sqsubset a_2 \\ (a_1, b_1 \sqcup b_2) & \text{if } a_1 = a_2 \\ (a_1 \sqcup a_2, \perp) & \text{if } a_1 \parallel a_2 \end{cases}$$

(when B has a bottom or A is a chain)

Powerset $\mathcal{P}(S)$

$$\sqsubseteq = \subseteq$$

$$\sqcup = \cup$$

(set S)

Function space $A \rightarrow B$

$$f \sqsubseteq g = \forall x \in A. f(x) \sqsubseteq g(x)$$

$$(f \sqcup g)(x) = f(x) \sqcup g(x)$$

(set A to join-semilattice B)

(map $K \rightarrow V$, V with bottom, where missing keys yield bottom is specially useful)

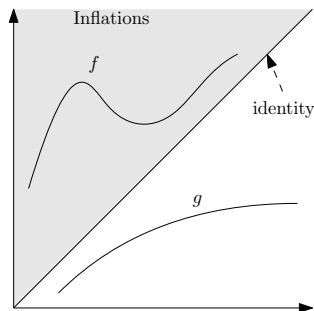
State-based CRDTs

- ▶ Operations applied locally
- ▶ State is a join-semilattice
- ▶ Replicas send the full state
- ▶ Replicas merge received state using join operator $\sqcup$
- ▶ Work in unreliable networks (loss, duplication, reordering)
- ▶ Sending full state ensures causal consistency

Update operations and state mutators

- ▶ State needs to evolve monotonically
 - when updates are locally invoked
 - when remote state is received and merged through join
- ▶ Update operations invoke a state mutator
- ▶ State mutator must be an *inflation*
 - (inflation) $x \sqsubseteq f(x)$
 - (strict inflation) $x \sqsubset f(x)$

Inflations vs monotonic functions



- ▶ Erroneously, many places say mutators need to be monotonic
- ▶ State mutators need to be inflations
- ▶ Being monotonic is not necessary nor sufficient
- ▶ f is an inflation but not monotonic
- ▶ g is monotonic but not an inflation

Trivial CRDTs – example: Grow only set $\text{GSet}\langle E \rangle$

$$\begin{aligned}\text{GSet}\langle E \rangle &= \mathcal{P}(E) \\ \perp &= \{\} \\ \text{add}_i(e, s) &= s \cup \{e\} \\ \text{elements}(s) &= s \\ \text{contains}(e, s) &= e \in s \\ s \sqcup s' &= s \cup s'\end{aligned}$$

- ▶ Trivial CRDT: state same as sequential datatype
- ▶ Add already an inflation, commutative and idempotent
- ▶ Merging replica states by set union
- ▶ Anonymous CRDT: no need for node ids in the state

Single-writer principle and named CRDTs

- ▶ Powerful strategy: single writer principle
 - state partitioned in several parts
 - each node updates a part dedicated exclusively to itself
 - state is joined by joining respective parts
- ▶ Unique node ids can be used to partition state
 - as a map from ids to parts
 - example: version vectors
- ▶ CRDTs that use node ids in the state are *named CRDTs*

Lamport (1974), **A New Solution of Dijkstra's Concurrent Programming Problem**
Enes, Almeida, and Baquero (2017), **The Single-Writer Principle in CRDT Composition**

State-based GCounter

$$\text{GCounter} = \mathbb{I} \rightarrow \mathbb{N}$$

$$\perp = \{\}$$

$$\text{inc}_i(m) = m\{i \mapsto m[i] + 1\}$$

$$\text{value}(m) = \sum \{m[j] \mid j \in \text{dom } m\}$$

$$m \sqcup m' = \{j \mapsto \max(m[j], m'[j]) \mid j \in \text{dom } m \cup \text{dom } m'\}$$

- ▶ State maps replica ids to integers
- ▶ inc increments self entry (i)
- ▶ Counter value obtaining by adding entries
- ▶ Join is pointwise max
- ▶ Similar in structure to a version vector

State-based PNCounter

$$\begin{aligned}\text{PNCounter} &= \text{GCounter} \times \text{GCounter} \\ \perp &= (\perp, \perp) \\ \text{inc}_i((p, n)) &= (\text{inc}_i(p), n) \\ \text{dec}_i((p, n)) &= (p, \text{inc}_i(n)) \\ \text{value}((p, n)) &= \text{value}(p) - \text{value}(n) \\ (p, n) \sqcup (p', n') &= (p \sqcup p', n \sqcup n')\end{aligned}$$

- ▶ Problem: dec is not natively an inflation
- ▶ Solved through a pair of GCounters (product composition)
 - increments and decrements tracked separately
 - counter value obtained as difference
- ▶ In practice, a single map to pairs is used

How to forget without tombstones

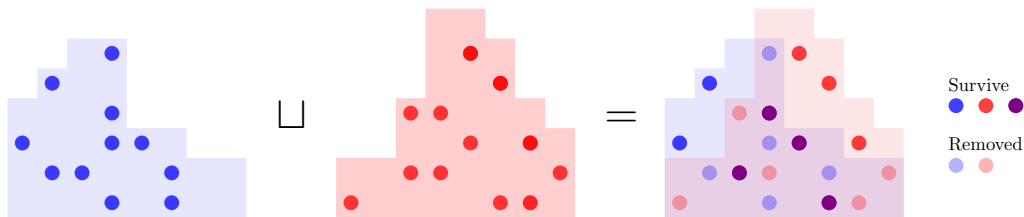
- ▶ Many times we want to remove things
 - example: set with add and remove
- ▶ But must always use inflations
 - cannot use single set for state and remove elements
- ▶ Using tombstones to mark removal makes state grow forever
- ▶ Example: two-phase set
 - allows add and remove
 - once removed, an element cannot be re-added
 - implementation with a pair of sets, to track adds and removes
 - state is always growing
- ▶ In a two-phase set removal is forever; what if we want to re-add, as usual?
- ▶ Tombstones can be avoided (in some cases) using *Causal CRDTs*

Causal CRDTs

- ▶ State has two components:
 - a *dot store*
 - a *causal context*
- ▶ The dot store (DS)
 - is a container for datatype-specific information
 - information tagged with unique event identifiers: *dots*
- ▶ The causal context (CC)
 - represents causal history: ids of all visible updates
 - encoded in a compact way – normally by a version vector

Info covered by causal context and not in the dot store has already been removed

Joining causal CRDTs



- ▶ The dot store that results from a join of x and y
 - has dots from x 's DS not covered by y 's CC
 - has dots from y 's DS not covered by x 's CC
 - has the dots present in both DSs
- ▶ The causal context upon a join is the join of causal contexts

Some dot stores

$$\begin{aligned}\text{DotSet} : \text{DS} &= \mathcal{P}(\mathbb{I} \times \mathbb{N}) \\ \text{DotFun}\langle V : \text{Lattice} \rangle : \text{DS} &= \mathbb{I} \times \mathbb{N} \multimap V \\ \text{DotMap}\langle K, V : \text{DS} \rangle : \text{DS} &= K \multimap V\end{aligned}$$

DotMap allows map CRDTs which embed CRDTs, including DotMaps themselves

$$\text{DotMap}\langle K_1, \text{DotMap}\langle K_2, \text{DotFun}\langle \mathcal{P}(E) \rangle \rangle \rangle$$

Joining causal CRDTs – for different dot stores

$$\text{Causal}\langle T : \text{DS} \rangle = T \times \text{CausalContext}$$

$$\sqcup : \text{Causal}\langle T \rangle \times \text{Causal}\langle T \rangle \rightarrow \text{Causal}\langle T \rangle$$

when $T : \text{DotSet}$

$$(s, c) \sqcup (s', c') = ((s \cap s') \cup (s \setminus c') \cup (s' \setminus c), c \cup c')$$

when $T : \text{DotFun}\langle _ \rangle$

$$(m, c) \sqcup (m', c') = (\{d \mapsto m[d] \sqcup m'[d] \mid d \in \text{dom } m \cap \text{dom } m'\} \cup \\ \{(d, v) \in m \mid d \notin c'\} \cup \{(d, v) \in m' \mid d \notin c\}, c \cup c')$$

when $T : \text{DotMap}\langle _, _ \rangle$

$$(m, c) \sqcup (m', c') = (\{k \mapsto v(k) \mid k \in \text{dom } m \cup \text{dom } m' \wedge v(k) \neq \perp\}, c \cup c') \\ \text{where } v(k) = \text{fst}((m[k], c) \sqcup (m'[k], c'))$$

Observed-remove set $\text{ORSet}\langle E \rangle$

$$\begin{aligned}\text{ORSet}\langle E \rangle &= \text{Causal}\langle \text{DotMap}\langle E, \text{DotSet} \rangle \rangle \\ \text{add}_i(e, (m, c)) &= (m\{e \mapsto \{(i, c[i] + 1)\}\}, c\{i \mapsto c[i] + 1\}) \\ \text{remove}_i(e, (m, c)) &= (\{e\} \triangleleft m, c) \\ \text{elements}((m, c)) &= \text{dom } m\end{aligned}$$

- ▶ Add replaces, for corresponding key, any set of dots by a new singleton
- ▶ Remove simply removes that key (domain subtraction)
 - no new event introduced in the causal context
- ▶ Previously defined join for the DotMap case:
 - keeps concurrently added elements
 - removes elements removed elsewhere if no dot survives

From sequential data types to CRDTs

Operation-based CRDTs

Pure operation-based CRDTs

State-based CRDTs

Delta State CRDTs

Comparison of approaches

Operation- vs State-based

Operation-based

- ▶ small messages; e.g., 'inc'
- ▶ does not work if messages lost or duplicated

State-based

- ▶ works under unreliable messaging
- ▶ but sends full states

Can we have the best of both worlds?

Delta State CRDTs

- ▶ State: like normal state-based CRDTs
- ▶ Messages: based on *delta-states*, that are states, hopefully small
- ▶ Essential difference: delta-mutators, which return delta states, to be
 - joined with current state
 - joined with other delta states, forming *delta-groups* d , to send in messages
- ▶ For each mutator m we can define a delta-mutator m^δ such that:

$$m(X) = X \sqcup m^\delta(X)$$

State-based

$$\begin{aligned} X_i &:= m(X_i) \\ X_i &:= X_i \sqcup X_j \end{aligned}$$

Delta-state-based

$$\begin{aligned} X_i &:= X_i \sqcup m^\delta(X_i) \\ X_i &:= X_i \sqcup d \end{aligned}$$

State vs delta-state – examples

State-based GCounter

$$\text{inc}_i(m) = m\{i \mapsto m[i] + 1\}$$

Delta-state-based GCounter

$$\text{inc}_i^\delta(m) = \{i \mapsto m[i] + 1\}$$

State-based ORSet $\langle E \rangle$

$$\text{add}_i(e, (m, c)) = (m\{e \mapsto d\}, c \cup d)$$

where $d = \{(i, \text{next}_i(c))\}$

$$\text{remove}_i(e, (m, c)) = (\{e\} \triangleleft m, c)$$

Delta-state-based ORSet $\langle E \rangle$

$$\text{add}_i^\delta(e, (m, c)) = (\{e \mapsto d\}, d \cup m[e])$$

where $d = \{(i, \text{next}_i(c))\}$

$$\text{remove}_i^\delta(e, (m, c)) = (\{\}, m[e])$$

Delta-mutator returns:

- ▶ for counter: map with just self-entry
- ▶ for set: pair where CC has dots representing element, plus new dot for add

CC representation for replica state:

- ▶ VV with extra dots / exceptions if non-causal propagation; normal VV if causal

Basic anti-entropy algorithm

durable state:

$X_i \in S$, CRDT state; initially $X_i = \perp$

volatile state:

$D_i \in S$, delta-buffer; initially $D_i = \perp$

on operation _{i} (m^δ)

let $d = m^\delta(X_i)$

$X_i \leftarrow X_i \sqcup d$

$D_i \leftarrow D_i \sqcup d$

on receive _{j,i} (d)

$X_i \leftarrow X_i \sqcup d$

$D_i \leftarrow D_i \sqcup d$

periodically

let $m = \text{choose}_i(X_i, D_i)$

for j **in** neighbors _{i} **do**

 send _{i,j} (m)

$D_i \leftarrow \perp$

- ▶ Transitive propagation to allow arbitrary topologies and causal consistency
- ▶ Fails to ensure causal consistency under message loss
- ▶ Too naive; delta-buffer easily tends to full state

Problems with naive anti-entropy

Naive transitive propagation causes much redundancy

- ▶ deltas re-propagated back to where they came from
- ▶ delta-group wholly joined to delta-buffer, even if mostly reflected in state

Solution

- ▶ avoid back-propagation of delta-groups; tagging origin
- ▶ remove redundant state in received delta-groups; how?

Optimal deltas and smart anti-entropy through join decompositions

Join-irreducible

$$x = \bigsqcup F \Rightarrow x \in F$$

Join decomposition

$$D \subseteq \mathcal{J}(L) \wedge \bigsqcup D = x$$

Irredundant join decomposition (IJD)

$$D' \subset D \Rightarrow \bigsqcup D' \sqsubset \bigsqcup D$$

Unique IJD for distributive lattices

$$\Downarrow x = \max\{r \in \mathcal{J}(L) \mid r \sqsubseteq x\}$$

Difference

$$\Delta(a, b) = \bigsqcup \{y \in \Downarrow a \mid y \not\sqsubseteq b\}$$

Optimal delta mutator

$$m^\delta(x) = \Delta(m(x), x)$$

- ▶ For most CRDTs, state is not merely join-semilattice, but a distributive lattice
- ▶ Birkhoff's representation theorem allows obtaining unique IJD and optimal deltas

Birkhoff (1937), **Rings of sets**

Enes et al. (2019), **Efficient Synchronization of State-Based CRDTs**

From sequential data types to CRDTs

Operation-based CRDTs

Pure operation-based CRDTs

State-based CRDTs

Delta State CRDTs

Comparison of approaches

Open vs closed systems

Operation-based

- ▶ assumes known set of participants
- ▶ dynamic joining or retiring doable but complex
- ▶ but two independent groups cannot merge
 - not considering naive impls keeping all history
 - without becoming effectively state-based

State-based

- ▶ suits unknown set of participants
- ▶ trivial dynamic joining, leaving or merging independent groups

State size

Operation-based

- ▶ uses power from delegating to reliable delivery
- ▶ can have small state, independent of number replicas; e.g., counter

State-based

- ▶ assumes worst case unreliable messaging
- ▶ normally some state component grows with number of replicas
 - very relevant for counter
 - less relevant for set

Partition tolerance

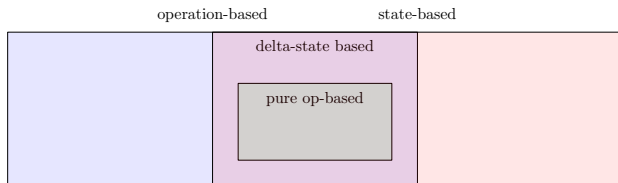
Operation-based

- ▶ transient cost (reliable delivery) linear with partition duration
 - unsuitable for long partitions
- ▶ transient cost can have component linear with number of replicas
 - but can be amortized over many objects in node

State-based

- ▶ each replica completely autonomous
- ▶ works well even under long partitions

Summary of comparison between CRDT approaches



	operation-based		state-based	
	general	pure-operation	standard	delta-state
reuse sequential ADTs	commutative ops		idempotent inflations	
open vs closed systems	fixed set of participants		dynamic independent groups	
partition tolerance	unsuitable for long partitions		good	
state size	independent of replicas		component linear with replicas	
metadata amortization	transparent (middleware)		explicit (container)	
message size	normally small	small	large	possibly small
messaging (usual)	causal broadcast		unreliable epidemic	
causal consistency	from messaging		for free	needs care
causal stability	useful	important	not available in general	

Final words

We came a long way from **ad hoc** optimistic replication

CRDTs offer a **principled approach** to optimistic replication

- ▶ Solved the “losing concurrent updates” problem
 - without requiring concurrency control or transactions
 - forgo read/write, use higher-level operations
- ▶ Like sequential datatypes, provide useful abstractions
 - difficult to obtain correct and efficient implementations
 - but need only be implemented by a few experts, can be used by many practitioners
- ▶ Have been successful in industry, to build large scale systems
 - with best consistency possible for AP: Causal Consistency + Convergence
 - with low response times even for large spatial span
- ▶ Now it is time to add transactional support
 - while reusing SQL database technology

CRDTs and Databases: A Hands-On Tutorial

Paulo Sérgio Almeida Nuno Faria José Pereira

INESC TEC and University of Minho

VLDB 2026



INESC TEC



Universidade do Minho
uminho.pt

This work is financed by National Funds through the Portuguese funding agency, FCT – Fundação para a Ciência e a Tecnologia; Paulo Almeida under the support UID/50014/2025 (DOI:10.54499/UID/50014/2025); Nuno Faria and José Pereira by CMU–Portugal project ADAPQO (DOI:10.54499/2024.12585.CMU).